Perfect! I've converted your transcript into **interview-ready, expanded notes** with emojis, diagrams, and clear explanations. Here's a structured version:

📁 ECS & Kubernetes Persistent Storage Notes for Interviews

1️⃣ ECS & EKS Setup Recap

- **ECS Cluster:** Core environment where containers run.
- **Node Types:**
  - **Managed Node Groups:** EC2 instances managed by AWS.
  - **Fargate Profiles:** Serverless compute for containers.
- **Load Balancer:** Exposes applications to the internet 🌐.

2️⃣ Why Storage Matters in Kubernetes

- Containers are ephemeral by default — **data is lost when pods stop**.
- Example use cases:
  - Persisting **user info in databases**
  - Retaining **uploaded files**

3️⃣ Kubernetes Storage Types

Type	Persistence	Use Case
Ephemeral	❌ Lost when pod stops	Cache, temp files
Persistent	✅ Survives pod lifecycle	Databases, uploads

- **Volumes:** Mounted into pods for read/write access.
- **Persistent Volumes (PV):** Kubernetes abstraction for persistent storage.

4️⃣ AWS EBS as Persistent Storage

- **EBS (Elastic Block Store):**

    - Block-level storage
    - Ideal for databases (MySQL, PostgreSQL)
    - Supports **high IOPS & throughput** ⚡
- **Dynamic provisioning in Kubernetes:**
    - Connect pods → EBS volumes via **EBS CSI Driver**

---

4. 1️⃣ CSI (Container Storage Interface)

- Acts as a **bridge** between Kubernetes & AWS.
- Integrates with Kubernetes objects:
    - **Persistent Volumes (PV)**
    - **Persistent Volume Claims (PVC)**
    - **Storage Classes**
- **Automatic lifecycle management** of volumes.

5️⃣ Security: IAM Roles for Service Accounts (IRSA)

- CSI driver requires AWS API permissions → handled via **IAM roles**.
- Steps:
    1. Create **IAM Role** with EBS permissions.
    2. Define **custom trust policy** → allows role to be assumed by a **Kubernetes service account**.
    3. Use **OIDC provider** (from EKS cluster) for authentication.
    4. Annotate service account → link to IAM role ✅
- **Result:** Least-privileged, secure access to AWS resources.

6️⃣ Stateful Applications in Kubernetes

- **Examples:** Databases, message queues, caching systems.
- **StatefulSets:**
    - Assign **stable, persistent identity** to each pod.
    - Create **dedicated PVCs per pod** using **volumeClaimTemplates**.

    - Allows **ordered, graceful scaling**.

6. 1️⃣ Key Benefits of StatefulSets

- Stable network identity
- Dedicated persistent storage per pod 💾
- Survives pod rescheduling/failure 🔄

7️⃣ Persistent Volume Claims (PVCs) Notes

- Deleting a pod **does NOT delete the volume**.
- Default **volume binding mode:** `WaitForFirstConsumer` ⏳
    - Volume created **only when a pod is scheduled**.
- **Access Mode for EBS:** `ReadWriteOnce`
    - Volume can only be mounted by **one node at a time**.

8️⃣ ECS Demo Workflow (Practical Steps)

1. **IAM Role Setup:**
    - Attach **Amazon EBS CSI Driver policy**
    - Define **trust policy** for CSI driver
2. **Install CSI Driver Add-on** in ECS cluster
3. **Configure IRSA:**
    - Annotate service account → link IAM role
4. **Deploy Storage:**
    - **Step 1:** Deployment → attach volume to a single pod
    - **Step 2:** StatefulSet → multiple pods with dedicated PVCs
5. **Observe Behavior:**
    - Volumes **remain attached** even after pod deletion/rescheduling

---

9️⃣ ASCII Diagram: Persistent Storage Flow

```
+---------------------+
```

```
|    Kubernetes Pod   |
|  (stateful app)     |
+---------+-----------+
          |
          | mounts
          v
+---------------------+
| Persistent Volume   |
|  (EBS Volume)       |
+---------------------+
          ^
          | provisioned dynamically
          |
+---------------------+
|  EBS CSI Driver     |
|  (Kubernetes-AWS)   |
+---------------------+
          ^
          |
+---------------------+
| IAM Role via IRSA   |
| (Secure access AWS) |
+---------------------+
```

🔑 Key Interview Takeaways

- EBS = preferred **persistent storage** for Kubernetes workloads in AWS.
- CSI driver = **bridge for dynamic provisioning & lifecycle management**.
- IRSA = **secure, least-privileged access** for storage operations.
- StatefulSets + PVCs = **resilient, stateful application deployments**.
- Default PVC mode = **WaitForFirstConsumer**, EBS access = **ReadWriteOnce**.

Great! Here's a detailed, **interview-ready version of your demo transcript** converted into structured notes with emojis, expanded explanations, and ASCII diagrams for clarity:

🗂️ IAM Role & OIDC Setup for EBS CSI in EKS – Notes

1️⃣ Context

- Goal: Enable **Amazon EBS volumes** to be used as persistent storage in Kubernetes pods via the **CSI driver**.
- Prerequisite:

- EKS cluster (`demo-eks-cluster`) already exists ✅
- At least **one node group** associated

2️⃣ Step 1: Create IAM Role with Custom Trust Policy

- Navigate to **IAM → Roles → Create Role**
- Select **Custom Trust Policy** (not predefined)

2.1 Update Sample Policy

1. **AWS Account ID:**
   - Found in **top-right corner** of AWS console
   - Replace placeholder in trust policy
2. **Region:**
   - Replace with cluster region (e.g., `us-east-2`)
3. **OIDC ID:**
   - From **EKS Cluster → Details → OIDC provider URL**
   - Extract ID and replace placeholder in policy
4. **Audience:**
   - `sts.amazonaws.com` (required to match the trust policy)

💡 **Purpose:** Authorizes the **service account in EKS** to assume this IAM role and use **EBS volumes via CSI driver**.

2.2 Attach Permissions

- Attach managed policy: **AmazonEBS_CSI_Driver_Policy** ✅
- Click **Next**, provide **Role Name & Description**
  - Suggested name: `Amazon_EBS_CSI_Driver_Role`
- Optional: Add **tags** for organization (e.g., `Name: x`)
- Click **Create Role**

3️⃣ Step 2: Configure Identity Provider (OIDC)

- Navigate to **IAM → Identity Providers → Add Provider**
- Select **OpenID Connect (OIDC)** as provider type

- Enter:
  - **OIDC Provider URL** (from EKS cluster details)
  - **Audience:** `sts.amazonaws.com` (matches trust policy)
- Add tags if desired for consistency
- Click **Add Provider**

💡 **Purpose:**

- Enables **service accounts in EKS** to authenticate to this IAM role
- Ensures secure, least-privilege access to AWS resources

---

4 Diagram: IAM Role & OIDC Flow

```
+---------------------------+
|  EKS Cluster Service      |
|  Account (Pod)           |
+------------+--------------+
             |
 Authenticates via OIDC
             v
+---------------------------+
|  IAM Role: EBS CSI Driver |
|  (Custom Trust Policy)    |
+------------+--------------+
             |
   Permissions granted to
             v
+---------------------------+
|  Amazon EBS Volumes       |
|  (Persistent Storage)     |
+---------------------------+
```

---

5 Key Points for Interviews

- **Custom Trust Policy:** Links EKS service account → IAM role
- **OIDC Identity Provider:** Authenticates pods to assume role

- **Managed Policy (AmazonEBS_CSI_Driver_Policy):** Grants EBS access
- **Tags:** Optional but help track roles and providers
- **Audience (`sts.amazonaws.com`):** Must match in both policy & provider

✅ Outcome:

- Pods in EKS cluster can dynamically provision **EBS volumes** using CSI driver securely.

Perfect! Here's a detailed, **interview-ready note version** of your EBS CSI driver installation demo, expanded with explanations, emojis, and ASCII diagrams:

📁 Installing & Configuring EBS CSI Driver in EKS – Notes

1️⃣ Context

- Goal: Install **Amazon EBS CSI Driver** on EKS cluster and configure **service account** to use the IAM role created earlier.
- Prerequisites:
  - EKS cluster exists (`demo-eks-cluster`) ✅
  - IAM role for EBS CSI driver created
  - OIDC identity provider configured

2️⃣ Step 1: Install EBS CSI Driver Add-on

1. Navigate to **EKS Cluster → Add-ons → Get more add-ons**
2. Search for **Amazon EBS CSI Driver**
3. Select the **latest version**
4. Associate IAM permissions:
   - Choose **IAM Role**
   - Select **the IAM role created earlier** for EBS CSI
5. Review & click **Create** ✅

💡 Outcome: Driver installed, ready to provision EBS volumes dynamically.

---

3 Step 2: Verify Service Account Creation

- Run command to check service accounts:

```
kubectl get serviceaccounts -n kube-system | grep ebs
```

- Expected output:

  ```
  ebs-csi-controller-sa
  ```

- **Purpose:** This service account is automatically created during driver installation.

---

4 Step 3: Annotate Service Account

- **Why:** Link Kubernetes service account → IAM role so pods can assume it.

1. Get **IAM Role ARN**:
   - Navigate to **IAM → Roles → EBS CSI Driver role → Copy ARN**
2. Annotate service account:

```
kubectl annotate serviceaccount ebs-csi-controller-sa \
  -n kube-system eks.amazonaws.com/role-arn=<IAM_ROLE_ARN>
```

- Verify annotation:

```
kubectl describe serviceaccount ebs-csi-controller-sa -n kube-system
```

- Check **Annotations field** → should show IAM role ARN

💡 Outcome: Service account can authenticate via IAM role to provision EBS volumes.

---

5 ASCII Diagram: EBS CSI Driver & IAM Role Flow

```
+---------------------------+
| Kubernetes Pod requesting |
| persistent storage        |
+------------+--------------+
             |
Authenticates via annotated
service account (ebs-csi-controller-sa)
             v
+---------------------------+
| IAM Role (EBS CSI Driver) |
| Permissions to manage EBS |
```

```
+------------+--------------+
             |
Dynamically provisions EBS volume
             v
+--------------------------+
| Amazon EBS Volume        |
| (Persistent Storage)     |
+--------------------------+
```

6️⃣ Key Points for Interviews

- **EBS CSI Driver:** Enables dynamic provisioning of EBS volumes for Kubernetes workloads
- **Service Account (`ebs-csi-controller-sa`):** Automatically created; must be annotated with IAM role ARN
- **Annotation:** Connects Kubernetes → IAM role → EBS permissions
- **Verification:** Use `kubectl describe serviceaccount` to confirm annotation
- **Result:** Pods can request persistent storage securely and dynamically

---

✅ **Takeaway:**

- Installing EBS CSI driver + annotating service account = allows Kubernetes workloads to **securely provision and use persistent EBS storage** in EKS using IAM-based authentication.

Perfect! Here's a **comprehensive, interview-ready note version** of your latest demonstration about **using the EBS CSI driver to provision volumes with PVCs**, expanded with explanations, emojis, and ASCII diagrams.

🗂️ Using EBS CSI Driver to Automatically Provision Volumes – Notes

1️⃣ Context

- Goal: Use **EBS CSI Driver** to dynamically provision **persistent storage** for pods in EKS via **Persistent Volume Claims (PVCs)**.
- Prerequisites:
    - EKS cluster with **managed node group** (Fargate not supported)

- EBS CSI driver installed ✅
- IAM role configured for CSI driver

---

2 Step 1: Create Persistent Volume Claim (PVC)

- **PVC Manifest Key Points:**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: demo-pvc
spec:
  storageClassName: gp2       # AWS default storage class
  accessModes:
    - ReadWriteOnce           # EBS limitation: attached to 1 node only
  resources:
    requests:
      storage: 1Gi            # Request 1GB volume
```

- **Storage Class Info:**
  - Use: `kubectl get storageclass` → should see `gp2`
  - **Binding Mode:** `WaitForFirstConsumer`
    - Volume not provisioned until **pod actually requests it**
- **Apply PVC:**

```
kubectl apply -f pvc-csi-ebs.yaml
kubectl get pvc
```

- **Observation:** PVC status = `Pending` → "waiting for first consumer"

💡 **Reclaim Policy:** `Delete` → deleting PVC deletes PV and EBS volume automatically

---

3 Step 2: Create Pod Using PVC

- **Pod Manifest Key Points:**

```
apiVersion: v1
kind: Pod
metadata:
```

```
  name: pod-with-pvc
spec:
  containers:
  - name: demo-container
    image: busybox
    command: ["echo", "Hello EBS"]
    volumeMounts:
    - mountPath: /opt
      name: demo-volume
  volumes:
  - name: demo-volume
    persistentVolumeClaim:
      claimName: demo-pvc
```

- **Apply Pod Manifest:**

  ```
  kubectl apply -f pod-with-pvc.yaml
  kubectl get pods
  ```

- PVC automatically **bound** → PV created by EBS CSI driver
- Verify PV:

  ```
  kubectl get pv
  kubectl describe pv <pv-name>
  ```

---

4 Step 3: Confirm EBS Volume in AWS

- Navigate to **EC2** → **EBS** → **Volumes**
- Check volume:
  - Size: 1 GB
  - Status: `in-use`
  - Attached to the **node running the pod**
- Cross-verify pod node IP → EC2 instance → storage tab

---

5 Step 4: Cleanup Process

1. **Delete Pod:**

   ```
   kubectl delete -f pod-with-pvc.yaml
   ```

- Pod removed
- PVC & PV remain (persistent) ✅

2. **Delete PVC:**

   `kubectl delete -f pvc-csi-ebs.yaml`

   - PV deleted automatically
   - EBS volume removed from AWS console

---

6 ASCII Diagram: PVC → PV → EBS Volume Flow

```
Pod requests storage
        |
        v
+-------------------+
| PersistentVolumeClaim |
|   demo-pvc          |
+---------+---------+
          |
          v
+-------------------+
| PersistentVolume  |
|   demo-pv         |
| Provisioned by    |
|  EBS CSI Driver   |
+---------+---------+
          |
          v
+-------------------+
| Amazon EBS Volume |
| Size: 1GB        |
| Status: In-use    |
| Attached to Node |
+-------------------+
```

7️⃣ Key Interview Takeaways

- **PVCs:** Abstract way for pods to request storage
- **Dynamic provisioning:** EBS CSI driver creates PV & EBS volume **on-demand**
- **Access Mode:** `ReadWriteOnce` → EBS limitation, only 1 node at a time
- **Binding Mode:** `WaitForFirstConsumer` → efficient resource use
- **Reclaim Policy:** `Delete` → automatic cleanup of PV & EBS volume
- **Important Note:** EBS CSI driver **does not work with Fargate**; use **managed node groups**

✅ **Outcome:** Pods get **secure, persistent storage**, dynamically provisioned & automatically cleaned up when PVC is deleted.

Excellent! Here's a **detailed, interview-ready note version** of your **StatefulSet + EBS CSI dynamic provisioning demo**, expanded with explanations, emojis, and ASCII diagrams:

📁 Dynamic Storage Provisioning with StatefulSets & EBS CSI – Notes

1️⃣ Context

- Goal: Automatically provision **persistent volumes (PV)** and **EBS volumes** for multiple pods using **StatefulSets**.
- Scenario: Manual PVC creation for each pod is inefficient for stateful workloads.
- Key Feature: **Volume Claim Templates** in StatefulSets let EKS automatically create PVCs per pod.

2️⃣ StatefulSet Manifest Key Points

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  replicas: 1               # initially 1 pod
  selector:
    matchLabels:
```

```
    app: web
serviceName: "web"
template:
  metadata:
    labels:
      app: web
  spec:
    containers:
    - name: nginx
      image: nginx
      volumeMounts:
      - name: www
        mountPath: /usr/share/nginx/html
volumeClaimTemplates:      # ✅ Automatic PVC creation
- metadata:
    name: www
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: gp2
    resources:
      requests:
        storage: 1Gi
```

Highlights:

- `volumeClaimTemplates` automatically provisions a **PVC for each pod**
- PVC → PV → EBS volume provisioning happens **dynamically**
- Each pod gets a **dedicated EBS volume**

---

3 Step 1: Deploy StatefulSet

1. Ensure no PVCs exist:

   ```
   kubectl get pvc
   ```

2. Apply StatefulSet manifest:

   ```
   kubectl apply -f statefulset-web.yaml
   ```

3. Verify pod:

   ```
   kubectl get pods -o wide
   ```

4. Check PVC:

   ```
   kubectl get pvc
   ```

   - New PVC created automatically
   - PVC status = `Bound`

5. AWS console (EC2 → EBS):
   - 1 GB volume attached to the node where the pod is running

---

4️⃣ Step 2: Scale StatefulSet

- Scale to **2 replicas**:

  ```
  kubectl scale statefulset web --replicas=2
  ```

- Outcome:
  - Second pod created on another node
  - New PVC created automatically
  - New EBS volume attached to the second node
- Scale to **3 replicas**:
  - Third pod created → dynamic PVC & EBS volume attached to first node

✅ **Observation:** Each pod gets a dedicated EBS volume automatically.

5️⃣ Step 3: Deletion Behavior

- StatefulSet deletion does **not delete PVCs** (data retention by design)
- Steps:

  ```
  kubectl delete statefulset web
  kubectl get pods          # none running
  kubectl get pvc           # PVCs still exist
  ```

- Manual cleanup of PVCs:

  ```
  kubectl delete pvc --all
  ```

- PVs and EBS volumes are deleted automatically
- Verify in AWS console: EBS volumes detached & removed

---

6 ASCII Diagram: StatefulSet + EBS CSI Dynamic Provisioning

```
  StatefulSet with N replicas
          |
          v
  +-------------------------+
  | Volume Claim Templates  |
  | Creates PVC per pod     |
  +------------+------------+
               |
               v
  +-------------------------+
  | Persistent Volume (PV)  |
  | Provisioned by EBS CSI  |
  +------------+------------+
               |
               v
  +-------------------------+
  | Amazon EBS Volume       |
  | 1GB attached to node    |
  +-------------------------+


Scaling StatefulSet → repeat above for each pod dynamically
```

---

7 Key Interview Takeaways

- **StatefulSet + Volume Claim Templates:** Automates PVC creation per pod
- **Dynamic Provisioning:** CSI driver provisions PV & EBS volume automatically
- **Access Mode:** `ReadWriteOnce` → volume attached to a single node
- **Scaling:** New pods → new PVCs → new PVs → new EBS volumes
- **Deletion:**
  - Pods deleted → PVC remains (persistent data)
  - Manual PVC deletion → PV & EBS deleted

- **AWS Console Verification:** EBS volume attached to the **node running the pod**

✅ **Outcome:** Simplifies **stateful workloads** in EKS with fully automated persistent storage management.

Perfect! Here's a **concise, interview-ready note version** of your **Amazon EFS lecture**, expanded with explanations, emojis, and comparison with EBS:

📁 Amazon EFS for Kubernetes Workloads – Notes

1️⃣ Context

- **Problem:** Amazon EBS is **block storage** → can only attach **one volume to one node at a time**.
- **Use case:** Multiple pods on different nodes (or Fargate) need **shared access** to the same data.
- **Solution: Amazon EFS (Elastic File System)** ✅

2️⃣ Key Features of Amazon EFS

- Fully managed, **serverless, elastic file storage**
- Behaves like a normal filesystem but in the cloud ☁️
- Can be **mounted by multiple pods on different nodes simultaneously**
- Pay **only for storage used**
- Ideal for:
    - CMS systems (WordPress, Drupal)
    - Developer tools (Jira, Git)
    - Shared notebooks (Jupyter)
    - Multi-user home directories

3️⃣ EBS vs EFS Comparison

Feature	EBS (Block Storage)	EFS (File Storage)
Access Mode	ReadWriteOnce (1 node/pod)	ReadWriteMany (multiple nodes/pods)
Mountable on Fargate?	❌ No	✅ Yes
Use case	Databases, high IOPS workloads	Shared content, multi-user workloads

Feature	EBS (Block Storage)	EFS (File Storage)
Volume Binding	WaitForFirstConsumer	Immediate
Scalability	Pre-provisioned	Elastic, scales automatically

💡 Key takeaway: **EFS = shared, multi-node access; EBS = single-node, high performance.**

4️⃣ EFS Setup in EKS/ECS

Steps:

1. **Create IAM role** with **Amazon EFS CSI Driver policy**
2. Use **IRSA (IAM Role for Service Accounts)** to link IAM role → CSI driver
3. Install **Amazon EFS CSI driver add-on** in the cluster
4. Ensure **EFS filesystem** exists and is available ✅
5. Define **StorageClass** referencing the EFS filesystem
6. Create **PersistentVolumeClaim**:
   - **Access mode:** `ReadWriteMany`
7. Deploy pods on **node groups** or **Fargate** to access the same volume concurrently

---

5️⃣ Key Differences from EBS

- Volume binding is **immediate** → no "wait for first consumer"
- Supports **multi-node, concurrent read/write access**
- Works with both **managed node groups** and **Fargate**
- EFS is perfect for workloads requiring **shared, concurrent access**

6️⃣ ASCII Diagram: EFS Multi-Pod Access

```
          +---------------------+
          | Amazon EFS Volume   |
          +---------+-----------+
                    |
       +------------+------------+
       |                         |
+-------------+           +-------------+
| Pod (Node 1)|           | Pod (Node 2)|
```

```
+-------------+           +-------------+
       |                         |
       |                         |
+-------------+           +-------------+
| Pod (Fargate)|           | Pod (Node 3)|
+-------------+           +-------------+
  Shared access across multiple pods
```

7️⃣ Interview Takeaways

- **EFS use case:** Shared workloads across nodes/Fargate
- **AccessMode:** `ReadWriteMany` → concurrent access
- **Binding:** Immediate → no scheduling dependency
- **EBS limitation:** single-node mount, not suitable for shared workloads
- **Setup difference:** EFS CSI uses **IRSA + CSI add-on** but does not rely on volume binding mode

✅ **Outcome:** Pods across **nodes or Fargate** can read/write simultaneously to the **same EFS volume**, enabling **true shared persistent storage** in Kubernetes.

Here's a **structured, interview-ready note version** of your **Amazon EFS CSI driver setup demo** with diagrams, emojis, and stepwise explanations:

📁 Amazon EFS CSI Driver Setup – Notes

1️⃣ Context

- **EBS limitation:** Only one node/pod can mount an EBS volume → **not suitable for shared workloads**
- **Solution:** Amazon EFS → shared file system accessible **concurrently by multiple pods** (nodes or Fargate)

2️⃣ Step 1: Create IAM Role for EFS

- Go to **IAM** → **Roles** → **Create Role**
- **Trusted entity:** AWS service → **ECS**

- **Authentication method: Pod Identity / Export Identity** (simpler than OIDC used for EBS)
- **Attach permissions:** `AmazonEFS_CSI_DriverPolicy` (AWS-managed)
- **Name:** Use consistent naming: `Amazon-EKS-EFS-CSI-Driver-Role`
- Add optional tags
- **Create role** ✅

💡 Difference from EBS:

- EBS uses **OIDC + IRSA**
- EFS uses **Pod Identity / Export Identity** → simpler workflow

---

3️⃣ Step 2: Install EFS CSI Driver

1. Go to **EKS Console** → **Cluster** → **Add-ons** → **Get More Add-ons**
2. Search for **Amazon EFS CSI driver**
3. **Install latest version**
4. **IAM association:** choose **Export Identity**
5. Select the role created in Step 1
6. Wait for installation → status = **Active** ✅

Result:

- CSI driver installed in the cluster
- Creates **service accounts** in `kube-system` namespace for EFS access

---

4️⃣ Step 3: Verify Service Accounts

```
kubectl get serviceaccounts -n kube-system
```

- Should see **2 new service accounts** linked to the EFS CSI driver
- These service accounts allow pods to **access EFS volumes**

5️⃣ Step 4: EFS File System Setup

- **Important:** CSI driver does **not create EFS file system** automatically
- EFS must already exist and be **network reachable**

Steps to prepare EFS:

1. Go to **AWS Console** → **EFS** → **Create File System**

2. Give it a **name**, e.g., `demo-x-filesystem`
3. Must be in **same VPC** as ECS cluster
4. Ensure **subnets** of node groups or Fargate profiles can reach EFS mount targets
5. Configure **security group**: allow **inbound NFS traffic on port 2049** (EFS requirement)
6. Confirm **mount targets** in all required subnets

💡 Tip: For Fargate pods, ensure subnets have network access to EFS mount targets

---

6 Summary of Setup

1. **IAM Role:** Export Identity → linked to CSI driver
2. **EFS CSI Driver:** Installed as cluster add-on
3. **EFS File System:** Existing, reachable, with proper security & subnets
4. Ready to use **EFS as persistent volume** in pods

---

7 ASCII Diagram: EFS CSI Driver Architecture

```
          +----------------------+
          | IAM Role: EFS Access |
          +----------+-----------+
                     |
                     v
       +----------------------------+
       | EFS CSI Driver Add-on      |
       | Creates service accounts   |
       +------------+---------------+
                    |
        +-----------+------------+
        |                        |
+-------------+            +-------------+
| Pod (Node1)|            | Pod (Fargate)|
+-------------+            +-------------+
        |                        |
        +-----------+------------+
                    |
```

```
        +----------------------+
        | Amazon EFS FileSystem |
        +----------------------+
Shared access across multiple pods/nodes/Fargate
```

8️⃣ Key Interview Takeaways

- **EFS vs EBS:**
  - EFS → multi-node, shared, concurrent access (`ReadWriteMany`)
  - EBS → single-node only (`ReadWriteOnce`)
- **Authentication:** EFS CSI driver uses **Export Identity / Pod Identity**
- **Driver setup:** Add-on in EKS → service accounts created automatically
- **Prerequisites:** Existing EFS file system with proper network & security

✅ **Outcome:** Cluster now supports **shared persistent storage** for pods across nodes and Fargate.

Here's a **well-structured, concise, and interview-ready note version** of your EFS multi-pod/multi-node/Fargate demo, including diagrams and key points:

🗂️ Amazon EFS CSI Driver – Multi-Pod and Fargate Demo

1️⃣ Context

- **Use case:** Shared storage across multiple pods **and nodes**, including Fargate
- **Limitation of EBS:** ReadWriteOnce → cannot be shared across multiple nodes or Fargate
- **Solution:** Amazon EFS → ReadWriteMany, concurrent multi-node access

---

2️⃣ Step 1: Storage Class

- **Manifest:** `kind: StorageClass`
- **API version:** `storage.k8s.io/v1`
- **Name:** `efs-sc`
- **Provisioner:** `efs.csi.aws.com`

- **Binding mode:** Immediate → PVC binds as soon as it's created
- Compare with **GP2/EBS:** WaitForFirstConsumer

Verify existing storage classes:

```
kubectl get sc
```

Apply storage class:

```
kubectl apply -f efs-sc.yaml
```

3️⃣ Step 2: Persistent Volume (PV)

- **Kind:** PersistentVolume
- **Name:** `efs-pv`
- **Storage class:** `efs-sc`
- **VolumeMode:** FileSystem
- **AccessMode:** `ReadWriteMany` → multiple pods can read/write concurrently
- **ReclaimPolicy:** Retain (optional: Delete)
- **VolumeHandle:** EFS file system ID (pre-created)

Verify PV creation:

```
kubectl get pv
```

4️⃣ Step 3: Persistent Volume Claim (PVC)

- **Kind:** PersistentVolumeClaim
- **Name:** `efs-claim`
- **StorageClass:** `efs-sc`
- **AccessMode:** `ReadWriteMany`
- **Size:** 5 GB

Apply PVC and verify binding:

```
kubectl apply -f efs-pvc.yaml
kubectl get pvc
```

✅ Status should be **Bound** → ready for use in pods

---

5 Step 4: Node Group Deployment

- **Deployment name:** `demo-deploy`
- **Replicas:** 3
- **Volume:** Mounted using `efs-claim` at `/data`
- **Workload:** Each pod writes to `/data/out1.txt` every 5 seconds

Apply deployment and verify pods:

```
kubectl apply -f demo-deploy.yaml
kubectl get pods
```

Verify shared writes:

```
kubectl exec -it <pod-name> -- cat /data/out1.txt
```

- File contains entries from **all pods** → confirms multi-pod, shared access

Scaling example:

```
kubectl scale deployment demo-deploy --replicas=5
```

- 2 new pods automatically start → all writing to the **same file** on EFS

---

6 Step 5: Fargate Deployment

- **Deployment name:** `demo-deploy-fargate`
- **Replicas:** 3
- **Profile:** Fargate
- **Volume:** Same `efs-claim` → mounted at `/data`
- **Behavior:** Pods initially Pending → nodes dynamically provisioned
- **Verify shared writes:**

```
kubectl exec -it <fargate-pod-name> -- cat /data/out1.txt
```

- EFS mounted → all pods (NodeGroup + Fargate) writing to **same shared file**

---

7 Step 6: Cleanup

- Delete deployments:

```
kubectl delete -f demo-deploy.yaml
kubectl delete -f demo-deploy-fargate.yaml
```

- Delete PVC:

```
kubectl delete pvc efs-claim
```

- Delete PV (optional)
- **EFS file system and storage class remain intact**

8️⃣ Key Takeaways

Feature	EBS	EFS
Access Mode	ReadWriteOnce	ReadWriteMany
Multi-node / Fargate	❌	✅
Volume creation	Automatic via CSI	File system must exist
Use case	Database, single pod storage	Shared workloads, multi-node apps
Binding mode	WaitForFirstConsumer	Immediate

9️⃣ ASCII Diagram – Multi-Pod + Multi-Node + Fargate EFS

```
        +-------------------+
        |  EFS FileSystem   |
        +---------+---------+
                  |
        +---------+---------+
        |                   |
+----------------+    +----------------+
| NodeGroup Pods |    |  Fargate Pods  |
|  /data/out1.txt|    | /data/out1.txt |
+----------------+    +----------------+
All writing concurrently → ReadWriteMany
```

✅ **Result:**

- Multiple pods across **node groups** and **Fargate** share the **same EFS volume**
- True multi-node, concurrent read/write access
- Supports both **persistent shared data** and **dynamic scaling**

Here's a **concise, structured, and interview-friendly note version** of your Application Load Balancer (ALB) lecture with EKS/ECS:

⚡ Application Load Balancer (ALB) with Amazon EKS/ECS

1️⃣ Overview

- **ALB Layer:** Operates at **Layer 7 (Application layer)** → HTTP/HTTPS routing
- **Use case:** Route traffic based on **hostnames**, **paths**, or **other HTTP headers**
- **Controller: AWS Load Balancer Controller** automates provisioning & configuration of ALBs and NLBs from Kubernetes resources (Ingress, Service)

2️⃣ History & Background

- Initially called **AWS ALB Ingress Controller**
- Developed by **Ticketmaster & CoreOS**
- Donated to **Kubernetes SIG AWS**
- Maintained by AWS + community contributors
- Bridges Kubernetes resources and AWS load balancers → creates ALB, listeners, target groups, and routing rules automatically

---

3️⃣ How It Works

1. Define a **Kubernetes Ingress resource** with proper annotations
2. Controller watches the Kubernetes API → creates ALB automatically
3. Sets up:
    - **Listeners:** typically port 80 (HTTP) & 443 (HTTPS)
    - **Target groups:** backend services/pods
    - **Routing rules:** path-based (e.g., `/participate`) or host-based (e.g., `app.example.com`)

- Works with both **EC2 nodes** and **Fargate** pods
- Can deploy ALB in **public** (internet-facing) or **private** subnets (internal apps)

---

4 ALB Traffic Modes

Mode	Description	Best Use Case
Instance	Traffic → NodePort on EC2 → pod	EC2 worker nodes
IP	Traffic → Pod IP directly, bypasses node port	Fargate or hybrid clusters

Demo choice: IP mode (required for Fargate)

---

5 Demo Scenario: Lucky Draw App

- **Services:**
  1. `participate-service` → handles submissions
  2. `submission-service` → displays submitted entries
- **ALB routing rules:**
  1. `/participate` → `participate-service`
  2. `/submissions` → `submission-service`
- **Result:** One ALB DNS → multiple paths → multiple services

---

6 ALB Setup Steps (Hands-On)

1. **IAM Setup**
   - Create **custom IAM policy & role**
   - Trust relationship → only **Load Balancer Controller service account** in `kube-system` namespace can assume it
2. **Kubernetes Service Account**
   - Create SA → annotate with IAM role
3. **Install AWS Load Balancer Controller**
   - Add **AWS EKS Helm repo**
   - Install controller using **Helm chart**
   - Apply required **CRDs**: TargetGroupBinding, IngressClassParams
4. **Deploy Services & Ingress**
   - Deploy 3 services (2 exposed via Ingress)
   - Define Ingress:

    - **ALB annotations:** schema (HTTP/HTTPS), subnet IDs, target type (`IP`), backend protocol (`HTTP`)
  - Controller provisions ALB automatically → routes traffic based on path rules

---

7️⃣ Key Benefits

- **Automation:** No manual ALB creation required
- **Path-based routing:** Multiple services under a single DNS
- **Supports Fargate:** IP mode allows direct pod routing
- **Flexible deployment:** Public or private subnets, multiple target groups

---

8️⃣ Diagram – ALB Routing Example

```
          +-----------------------+
          |       ALB (IP mode)   |
          +---------+-------------+
                    |
          +---------+---------+
          |                   |
+----------------+     +----------------+
| /participate   |     | /submissions   |
| participate-svc|     | submission-svc |
+----------------+     +----------------+
Pods running on NodeGroup and Fargate
```

✅ **Summary:**
The AWS Load Balancer Controller integrates ALB into EKS/ECS, automates listener & target group management, supports **path-based routing**, **Fargate**, and **multi-service deployments** with minimal manual configuration.

Here's a **structured summary** of your demonstration for **creating IAM resources for ALB Ingress Controller in EKS/ECS**:

⚡ IAM Setup for AWS Load Balancer Controller (ALB Ingress) in EKS

1 Create IAM Policy

1. Navigate to **IAM → Policies → Create Policy**
2. Switch to **JSON tab** (predefined JSON with all required permissions)
3. Paste the JSON into the editor → **Next**
4. Give policy a **name:** `AWS Load Balancer Controller IAM Policy`
5. Optionally add tags (e.g., `Name: x`)
6. Click **Create policy** → now a **customer-managed policy**

   This policy contains all permissions needed for ALB Ingress Controller to manage AWS resources.

---

2 Create IAM Role

1. Go to **IAM → Roles → Create Role**
2. Select **Trusted Entity → Web Identity**
3. Choose the **OIDC provider** created for your EKS cluster
4. Audience: `sts.amazonaws.com`
5. Attach the previously created policy → **Next**
6. Name the role: `Amazon ECS Load Balancer Controller Role`
7. Optionally add tags (e.g., `Name: ECS`) → **Create role**

---

3 Edit Trust Policy

1. Open **Role → Trust relationships → Edit trust policy**
2. Add a condition specifying the **service account** allowed to assume the role:

```
"Condition": {
    "StringEquals": {
        "oidc.eks.<region>.amazonaws.com/id/<OIDC_ID>:sub":
"system:serviceaccount:kube-system:aws-load-balancer-controller"
    }
```

```
}
```

Ensures that **only the ALB Controller service account** in the `kube-system` namespace can assume this role.

3. Click **Update Trust Policy**

4️⃣ Create Service Account in Kubernetes

- Create a service account in `kube-system` namespace:

  ```
  kubectl create serviceaccount aws-load-balancer-controller -n kube-system
  ```

- Annotate it with the IAM role ARN:

  ```
  kubectl annotate serviceaccount aws-load-balancer-controller \
    -n kube-system
  eks.amazonaws.com/role-arn=arn:aws:iam::<ACCOUNT_ID>:role/AmazonECSLoadBalancerControllerRole
  ```

✅ **Result:**

- IAM policy ✅ created
- IAM role ✅ created with trust policy restricting access to ALB Controller
- Kubernetes service account ✅ linked to IAM role
- Ready to install **AWS Load Balancer Controller** using Helm in the EKS cluster

---

Here's a **structured summary** of your demonstration for **deploying the AWS Load Balancer Controller (ALB Ingress) in EKS**:

⚡ Deploying AWS Load Balancer Controller in EKS

1️⃣ Create and Annotate Service Account

1. Create a service account in `kube-system` namespace:

   ```
   kubectl create serviceaccount aws-load-balancer-controller -n kube-system
   ```

2. Annotate the service account with the IAM role ARN:

   ```
   kubectl annotate serviceaccount aws-load-balancer-controller \
   ```

```
  -n kube-system
eks.amazonaws.com/role-arn=arn:aws:iam::<ACCOUNT_ID>:role/AmazonECSLoadBalancerControllerRole
```

3. Verify annotation:

```
kubectl describe serviceaccount aws-load-balancer-controller -n kube-system
```

This allows the ALB controller pods to assume the IAM role with necessary permissions.

---

2 Install AWS Load Balancer Controller via Helm

1. Add Helm repository (if not already added):

```
helm repo add eks https://aws.github.io/eks-charts
```

```
helm repo update
```

2. Install the controller with Helm:

```
helm upgrade -i aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=demo-eks-cluster \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=us-east-2 \
  --set vpcId=<VPC_ID>
```

3. Verify installation:

```
helm list -n kube-system
```

Status should be `deployed`.

---

3 Install Custom Resource Definitions (CRDs)

1. Download CRDs YAML from AWS repository:

```
https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/master/docs/install/crds.yaml
```

2. Apply the CRDs:

```
kubectl apply -f crds.yaml
```

3. Verify ingress class:

```
kubectl get ingressclass
```

You should see an entry for `alb`.

4️⃣ Verify Controller Deployment

1. Check deployment:

   ```
   kubectl get deployments -n kube-system aws-load-balancer-controller
   ```

   Status should show `READY`.

2. Check pods:

   ```
   kubectl get pods -n kube-system | grep aws
   ```

   You should see **2 controller pods** running.

✅ Outcome

- Service account ✅ created and linked to IAM role
- Helm chart ✅ deployed for AWS Load Balancer Controller
- CRDs ✅ installed for target group bindings and ingress class
- Controller pods ✅ running in `kube-system`

The EKS cluster is now ready to use **Application Load Balancer (ALB) as an ingress controller** for your workloads.

Here's a **clear, structured summary** of your demonstration showing how the **ALB Ingress Controller works with EKS**:

⚡ Verifying ALB Ingress Controller in EKS

1️⃣ Deploy the Demo Application

- Namespace and resources created via manifest:
  - **1 Redis StatefulSet**
  - **2 Deployments** (`participate` and `submissions`)
  - **3 ClusterIP Services** (internal)
- **Service details:**
  - **Participate Service**: container port `5000`, service port `80`
  - **Submissions Service**: container port `5000`, service port `80`
- Apply manifest:

```
kubectl apply -f demo-app.yaml
```

---

2 Deploy the Ingress Resource

- Manifest defines **demo-alb-ingress**:
  - Uses **Alb Ingress class**
  - **Annotations**:
    - Internet-facing load balancer
    - Target type: IP
    - Protocol: HTTP
    - Public subnet IDs
  - **Path-based routing rules**:
    - `/participate` → participate service
    - `/submissions` → submissions service
- Apply ingress:

  ```
  kubectl apply -f demo-alb-ingress.yaml
  ```
- Verify:

  ```
  kubectl get ingress -n <namespace>
  ```
  - Shows DNS name of automatically provisioned ALB
  - ALB takes ~60–90 seconds to become active

---

3 Test the Application

1. Access **Participate service**:

   ```
   http://<ALB_DNS>/participate
   ```
   - Submit sample entries: e.g., Kulbushan Meyer, Yogesh Raja
   - Pod name handling the request is displayed
2. Access **Submissions service**:

   ```
   http://<ALB_DNS>/submissions
   ```
   - Confirms that submitted records are displayed
   - Matches pod names → ensures path-based routing works correctly

✅ Outcome: Traffic routed correctly to multiple services using a **single ALB DNS**.

4️⃣ Clean Up

- Delete ingress resource → automatically removes ALB:

  ```
  kubectl delete ingress demo-alb-ingress -n <namespace>
  ```

- Delete demo application resources:

  ```
  kubectl delete -f demo-app.yaml
  ```

- ALB disappears from AWS console automatically.

✅ Key Takeaways

- The **ALB Ingress Controller** automatically provisions an Application Load Balancer.
- Path-based routing allows multiple services to share a **single DNS**.
- Controller handles backend routing without manual ALB configuration.
- Works for EC2 and Fargate workloads.
- Demonstrates end-to-end integration: **Ingress → ALB → Service → Pods**.

Here's a **concise breakdown** of your lecture on **Cluster Autoscaler with EKS managed node groups**:

⚡ Cluster Autoscaler in EKS

1️⃣ Background: Auto Scaling vs. Cluster Autoscaler

- **Managed Node Groups** use an **Auto Scaling Group (ASG)**.
  - ASG maintains **desired node count**.
  - Without Cluster Autoscaler, ASG **does not scale based on pod demand**.
- **Cluster Autoscaler (CA)** dynamically adjusts nodes based on pod scheduling:
  - **Scale up**: When pods are pending due to insufficient resources.
  - **Scale down**: When nodes are underutilized and can be safely drained.
- Works by **simulating node additions/removals** before applying changes.
- Runs inside Kubernetes as a **deployment** in `kube-system` namespace.
  - Uses **leader election**; only one replica acts at a time.
  - Respects ASG min/max limits.

---

2 How Cluster Autoscaler Works on AWS

- Managed node groups:
  - Backed by **EC2 Auto Scaling Groups**.
  - Automatically tagged for **CA discovery**.
  - Support **graceful node draining** during scale down.
- For **mixed instance types**, CA uses the first instance type for simulation.
- Integrated with EKS for **simplified lifecycle management**:
  - In-place upgrades
  - Health replacement
  - ASG discovery via tags

---

3 Setting Up Cluster Autoscaler

Step 1: Create IAM Policy

- Provides permissions for:
  - **EC2**
  - **EC2 Auto Scaling**
  - **ECS (if needed)**
- Ensures the Cluster Autoscaler can scale nodes securely.

Step 2: Create IAM Role with OIDC

- Uses **Web Identity Federation**.
- Associates the role with a **Kubernetes service account**:
  - Only CA can assume this role.
- Secure and follows **best practices** for least privilege.

Step 3: Deploy Cluster Autoscaler

- Use **auto discovery method**:
  - CA detects eligible ASGs automatically via tags.
- No manual ASG configuration needed.
- Deployed as a **Kubernetes deployment** in the cluster.

4️⃣ Demonstration Scenarios

Scaling Up

1. Start with **2 nodes**.
2. Increase deployment replicas from 2 → 10.
3. Pending pods trigger **cluster autoscaler**.
4. ASG scales up **gradually to max 5 nodes**.
5. All pods scheduled and running across new nodes.

Scaling Down

1. Reduce deployment replicas back to 2.
2. Nodes become **underutilized**.
3. CA performs **graceful termination**:
    - Marks idle nodes as **unschedulable**.
    - Drains remaining pods.
    - Adjusts ASG desired capacity.
4. Cluster returns to **2 active nodes**.

Note: Node removal has slight delays due to **ASG cooldown periods**.

✅ Key Takeaways

- CA bridges the gap between **pod demand** and **node supply**.
- Handles **dynamic cluster resizing** automatically.
- Integrated with **EKS managed node groups** and ASGs.
- Supports **safe, graceful scale down** to avoid pod disruption.
- Observing scale-up and scale-down in real time reinforces **auto scaling concepts**.

Next step in your demos: **Create the IAM role with OIDC trust** and deploy the Cluster Autoscaler to see it in action.

Here's a **step-by-step summary** of your demonstration on creating IAM resources for the Cluster Autoscaler in EKS:

⚡ Cluster Autoscaler IAM Setup in EKS

1️⃣ Overview

- Goal: Create IAM **policy** and **role** to allow the **Cluster Autoscaler** to scale EKS nodes.
- Reference: Official AWS Cluster Autoscaler GitHub documentation
- Focus: **AWS-specific setup**, ignoring other cloud providers.

2️⃣ Step 1: Create IAM Policy

1. Navigate to **IAM → Policies → Create Policy**.
2. Switch to **JSON tab**.
3. Paste the **full-feature policy JSON** from the official documentation:
    - Grants access to:
        - **EC2**
        - **EC2 Auto Scaling**
        - **ECS**
4. Click **Next** and provide:
    - Name: `AWS Autoscaler Controller IAM Policy`
    - Optional description and tags (e.g., `Name: ECS`)
5. Click **Create Policy**.
6. Verify that the policy appears as a **customer-managed policy**.

3️⃣ Step 2: Create IAM Role

1. Navigate to **IAM → Roles → Create Role**.
2. Choose **Web Identity** as the trusted entity.
3. Select the **OIDC provider** for your EKS cluster.
4. Set **Audience**: `sts.amazonaws.com`
5. Attach the **IAM policy** created in Step 1 (`AWS Autoscaler Controller IAM Policy`).
6. Provide a name for the role:
    - `ECS Cluster Autoscaler`

7. Optional: Add tags (e.g., `Name: ECS`).
8. Click **Create Role**.

4️⃣ Step 3: Update Trust Relationship

1. Locate the role `ECS Cluster Autoscaler` in IAM.
2. Go to **Trust Relationships → Edit Trust Policy**.
3. Modify the policy to include a condition for the specific **service account**:

```
"Condition": {
    "StringEquals": {
        "sts:Sub": "system:serviceaccount:kube-system:cluster-autoscaler"
    }
}
```

4. Ensure the **service account name** matches the one you will use in the Kubernetes manifest.
5. Click **Update Trust Policy**.

✅ Outcome

- IAM **policy** and **role** are ready.
- The **Cluster Autoscaler** service account in Kubernetes can securely assume this role.
- Ready for **deployment of the Cluster Autoscaler** in your EKS cluster.

Here's a clear **step-by-step summary** of your demonstration showing the **Cluster Autoscaler in action on Amazon EKS**:

⚡ Cluster Autoscaler Demo on EKS

1️⃣ Overview

- Goal: Deploy Cluster Autoscaler with **auto discovery**.
- Auto discovery allows the autoscaler to **automatically detect the EKS node group ASGs** instead of manually specifying them.
- Cluster Autoscaler dynamically scales nodes based on **unschedulable pods** or **underutilized nodes**.

---

2 Download & Edit Deployment YAML

1. Navigate to **Cluster Autoscaler GitHub repo** → Auto Discovery YAML.
2. Download the YAML (`cluster-autoscaler-autodiscover.yaml`) to your local client.
3. Modify two sections:

a) Service Account Annotation

- Add the IAM role ARN so Kubernetes knows which role the service account should assume:

```
annotations:
  eks.amazonaws.com/role-arn: <Cluster-Autoscaler-IAM-Role-ARN>
```

- Service account name: `cluster-autoscaler`

b) Auto Discovery

- Update deployment args to include your cluster name:

```
--nodes=2:5:demo-eks-cluster
```

- Ensures the autoscaler targets the correct Auto Scaling Group.

---

3 Apply YAML and Verify

```
kubectl apply -f cluster-autoscaler-autodiscover.yaml
kubectl get pods -n kube-system
kubectl describe pod <cluster-autoscaler-pod> -n kube-system
```

- Pod should be **Running**.
- Environment variables confirm correct **AWS region** and **IAM role**.

---

4 Test Auto Scaling

a) Initial Deployment

- Deploy a test application:

```
replicas: 1
resources:
  cpu: 300m
  memory: 300Mi
```

- Existing cluster has 2 nodes.

b) Scale Up Pods

1. Scale deployment to **5 replicas**:

```
kubectl scale deployment cluster-autoscaler-demo --replicas=5
```

- Pods fit on existing nodes.

2. Scale to **10 replicas**:

```
kubectl scale deployment cluster-autoscaler-demo --replicas=10
```

- Several pods remain **Pending** due to insufficient CPU.
- Cluster Autoscaler triggers **scale-up**:
    - New node is provisioned (e.g., node ending with `19.24`)
    - Pending pods are scheduled on new nodes.

3. Scale to **20 replicas**:

- Multiple pods Pending.
- Cluster Autoscaler adds nodes **incrementally**, respecting the max of **5 nodes**.
- Once nodes are ready, pods are scheduled and running.

---

5 Test Scale Down

1. Scale deployment back to **2 replicas**:

```
kubectl scale deployment cluster-autoscaler-demo --replicas=2
```

2. Extra nodes are **not removed immediately** (cooldown/graceful termination):
    - Nodes first marked **SchedulingDisabled**
    - Pods drained
    - Node marked **NotReady**
    - Node removed from cluster
3. After some minutes, only 2 nodes remain, matching active pods.

---

6 Monitor Logs

```
kubectl logs -f <cluster-autoscaler-pod> -n kube-system
```

- Logs indicate **scale-up** and **scale-down events**.
- Shows which nodes are being added or removed.

✅ Outcome

- Cluster Autoscaler **automatically scales nodes up** when pods are pending due to resource constraints.
- **Scales down nodes gracefully** when underutilized.
- Works seamlessly with EKS managed node groups using **auto discovery**.
- No manual intervention required for node scaling; cluster adjusts dynamically based on pod demand.

Here's a concise breakdown of your **Amazon ECR repository creation demonstration**:

⚡ Creating an Amazon ECR Repository

1️⃣ Overview

- **Amazon Elastic Container Registry (ECR)** is AWS's service for storing and managing container images.
- Each ECR **registry** can contain multiple **repositories**, which hold different container images and tags for applications.

2️⃣ Steps to Create a Repository

a) Navigate to ECR

- Open the **AWS Management Console → ECR service**.
- Ensure you are in the desired **region** (e.g., Ohio, `us-east-2`).

b) Create a New Repository

1. Click **Create repository**.
2. Repository naming options:
    - **Simple name**: `myapp`
    - **Namespaced format**: `demo/welcome-app` (recommended for organization)
3. **Tag mutability**:
    - **Mutable**: Can overwrite existing image tags
    - **Immutable** (recommended): Prevents overwriting, ensures version control
4. **Encryption settings**:

    - Default: **AES-256** (industry standard)
    - Optionally: AWS KMS managed keys (AWS-managed or Customer-managed)
5. **Image scanning** (optional, now deprecated):
    - Can scan images for vulnerabilities

c) Finalize

- Click **Create repository**
- Repository is created with:
    - Name: `demo/welcome-app`
    - URI: `<registry-url>/demo/welcome-app:<tag>` (e.g., `v1`, `latest`)
    - Mutability: Immutable
    - Encryption: AES-256

3️⃣ Post-Creation Management

- View repository summary
- Push and manage images
- Edit repository settings
- Set permissions
- Delete repository if needed

Note: By default, ECR repositories are **private**. Public repositories are possible, but private repos are preferred in enterprise environments for security.

✅ Outcome: You now have a private ECR repository ready to store container images for your applications.

Here's a structured summary of your **Amazon ECR authentication and image push demonstration**:

⚡ Working with Amazon ECR from Ubuntu

1️⃣ Overview

- **Amazon ECR** stores container images securely.
- Access requires **authentication**, typically via the AWS CLI.

- Pushing images involves:
    1. Logging into ECR
    2. Pulling or tagging images
    3. Pushing them to the repository

---

2 Prerequisites

- Ubuntu machine with:
    - **AWS CLI** installed
    - **Docker** installed
- IAM user with **ECR permissions** (full access recommended for this demo)
- Private ECR repository already created (e.g., `demo/welcome-app`)

---

3 Grant ECR Permissions to IAM User

1. Navigate to **IAM → Users → Select your user → Add permissions**
2. Choose **Attach policies directly**
3. Search for **Container Registry** policies
4. Select **AmazonEC2ContainerRegistryFullAccess**
5. Confirm and attach the policy

IAM user now has full ECR access

---

4 Authenticate Docker with ECR

1. Copy login command from **View Push Commands** in ECR console:

   ```
   aws ecr get-login-password --region <region> | docker login --username AWS --
   password-stdin <ECR_registry_URL>
   ```

2. Run it on the Ubuntu machine
3. Verify:

   ```
   Login Succeeded
   ```

---

5 Pull Existing Public Images

- Pull images from public repository:

```
docker pull <image>:v1
docker pull <image>:v2
docker pull <image>:v3
```

- Check images:

```
docker image list
```

---

6 Tag Images for ECR

- Format:

```
docker tag <local-image>:<tag> <ECR_registry_URL>/demo/welcome-app:<tag>
```

- Examples:

```
docker tag welcome-app:v1 <ECR_registry_URL>/demo/welcome-app:v1
docker tag welcome-app:v2 <ECR_registry_URL>/demo/welcome-app:v2
docker tag welcome-app:v3 <ECR_registry_URL>/demo/welcome-app:latest
```

---

7 Push Images to ECR

1. Logout to test authentication:

```
docker logout <ECR_registry_URL>
```

2. Attempt to push (will fail without login)
3. Login again using AWS CLI token
4. Push images:

```
docker push <ECR_registry_URL>/demo/welcome-app:v1
docker push <ECR_registry_URL>/demo/welcome-app:v2
docker push <ECR_registry_URL>/demo/welcome-app:v3
docker push <ECR_registry_URL>/demo/welcome-app:latest
```

---

8 Verify in AWS Console

- Navigate to **ECR → demo/welcome-app**
- Check:
    - Versions: `v1`, `v2`, `v3`, `latest`
    - Image size
    - Push timestamp

    - URI: `<ECR_registry_URL>/demo/welcome-app:<tag>`
- Optional: Initiate **vulnerability scan**
- Optional: Enable **cross-region replication** (not enabled in demo)

✅ Outcome: The Ubuntu machine is authenticated, and multiple versions of the Welcome App image are pushed to ECR. Images are now ready for use in ECS, EKS, or any other containerized environment.

Here's a structured summary of your **Amazon ECR + ECS deployment demonstration**:

⚡ Deploying a Kubernetes Application on ECS from Amazon ECR

1️⃣ Overview

- Pull images from **Amazon ECR** instead of GitHub Container Registry.
- Deploy using **Kubernetes manifests** on an ECS cluster.
- IAM roles attached to ECS nodes handle **secure image pull** automatically.

2️⃣ Prepare Deployment Manifest

1. Copy the existing Kubernetes deployment manifest (previously using GitHub images).
2. Update the `image` field to point to the **ECR image URI**:

   `<account-id>.dkr.ecr.<region>.amazonaws.com/demo/welcome-app:v3`
3. Save the updated file as:

   `deployment-welcome-app-from-ecr.yaml`

Note: No image pull secrets are needed for ECR if the node IAM role has read access.

3️⃣ Verify IAM Role for ECS Nodes

- Navigate to **IAM → Roles**.
- Open the role associated with the ECS node group.
- Confirm it has the **AmazonEC2ContainerRegistryReadOnly** policy.
- This grants **EC2 instances** permission to pull images from ECR.

EC2 instances automatically assume this role, providing secure access to private ECR repos.

4️⃣ Deploy the Application

1. Attempt to apply the deployment:

   ```
   kubectl apply -f deployment-welcome-app-from-ecr.yaml
   ```

2. If the namespace doesn't exist:

   ```
   kubectl create namespace demo
   kubectl apply -f deployment-welcome-app-from-ecr.yaml
   ```

3. Verify pods:

   ```
   kubectl get pods -n demo
   ```

4. Describe a pod to confirm the image was pulled:

   ```
   kubectl describe pod <pod-name> -n demo
   ```

   - The pod uses the **ECR image URI**
   - No explicit credentials are configured in the manifest

5️⃣ Key Points

- **IAM roles for ECS nodes** allow pulling private ECR images without secrets.
- Simplifies deployment and follows **AWS best practices**.
- Works seamlessly for private organizational images stored in ECR.
- Reduces manual authentication overhead.

✅ Outcome: The Welcome App is successfully deployed on ECS using a private ECR image, leveraging node IAM roles for authentication.